

Foundations of Data Science for Biologists

Flow control and functions Python

BIO 724D

2026-MAR-17

Instructors: Greg Wray and Kayla Wilhoit

Learning objectives for today

Delve deeper into **control flow** and **functions** in Python

Become familiar with Python **functional programming** idioms

Learn about **exception handlers**

Learn about **iterators** and **generators**

Become familiar with the **itertools** and **pathlib** modules

Control flow

Control flow in Python

Standard `if...else`, `for`, and `while` control flow structures

It also provides the `match...case` structure for matching patterns

Conditional assignment, useful for situations where there are simple alternatives

Functional programming idioms that avoid explicit loops

Error handling using `try...except` and `raise`

Iterator and generator retrievals using `next` and `yield`

Special control flow statements

Three special statements can be included in control flow structures

None are necessary (work-arounds exist), but they clarify logic and are more readable

pass: a valid statement that does nothing

Useful as a placeholder for later work (blocks must contain at least one statement)

continue: proceeds directly to the next iteration of a loop or next evaluation in an if structure

Useful if a computationally intensive step doesn't need to execute each iteration

Ignores all subsequent statements in the code block for the *current iteration only*

break: completely exits the current loop and proceeds to the next statement

Useful when searching for a single specific item

Only works with loop structures; ignores *all* subsequent statements and conditions

if structures

Zero or one executes

```
if <condition> :      # conditional
    <statement(s)>
```

```
if <condition> :      # 2 conditionals
    <statement(s)>
elif <condition> :
    <statement(s)>
```

Order matters! (evaluation "short-circuits")

elif clauses require a condition

No limit on **elif** clauses

else clause guarantees 1 execution

Do not include a condition for an **else** clause

Exactly one executes

```
if <condition> :      # alternates
    <statement(s)>
else :
    <statement(s)>
```

```
if <condition> :      # alternatives
    <statement(s)>
elif <condition> :
    <statement(s)>
else :
    <statement(s)>
```


Multiple conditions

When testing multiple conditions, order matters for two reasons

Logic

- Conditions are evaluated in the order they are encountered

- The first conditional that evaluates to True ends the if / elif part of the structure

- Remember: the code block of *at most* one condition is evaluated

Efficiency

- Conditions are evaluated in the order they are encountered

- Test conditions in the order of expected frequency to minimize execution time

Conditional assignment

Python provides special `if` / `else` structure for conditional assignment

Syntax is simpler and often easier to read, but reserve for simple conditions

Known as **conditional assignment** or **ternary operator**

Normal `if` / `else` structure used for assignment:

```
if Ne > 100_000 :  
    pop_size = 'large'  
else :  
    pop_size = 'small'
```

Syntax for conditional assignment (note: the `else` clause is required):

```
pop_size = 'large' if Ne > 100_000 else 'small'
```


for loops

For loops use an iterable to determine the number of times the loop executes

The loop structure automatically updates the loop counter

Nesting for loops is a common way to systematically query a matrix or data frame

```
for <var> in <iterable> :                # simple loop
    <statement(s)>
```

```
for <var> in <iterable> : <statement>      # one-line (1 simple statement)
```

```
for <var> in <iterable> :                # loop followed by completion block
    <statement(s)>
else :                                  # always executes unless break/error
    <statement(s)>
```

Using ranges with for loops

The `range()` function provides a simple way to create custom iterators

Syntax:

```
for <var> in range(i, j, k) :    # can take 1, 2, or 3 arguments (see below)
    <statement(s)>
```

Examples:

```
for i in range(6) :                # iterates 6 times, i takes values 0 to 5
for i in range(2, 6) :              # iterates 4 times, i takes values 2 to 5
for i in range(0, 6, 2) :           # iterates 3 times, i takes values 0, 2, 4
```

while loops

While loops repeatedly execute a code block until a condition evaluates to **False**

Will not execute even once if the condition evaluates to **False** on first iteration

Risks infinite looping if the condition never evaluates to **False**

Always include a mechanism to ensure termination (see next slide)

Not generally recommended: only use if no other options work well

```
while <condition> :                # simple loop
    <statement(s)>
```

```
while <condition> :                # loop followed by completion block
    <statement(s)>
finally :
    <statement(s)>                # always executes
```


Ending while loops

Best practice with while loops: include a mechanism to guarantee exiting the loop

Strategy: set up conditions that *inevitably* trigger a `break` statement

Many mechanisms exist; 2 common examples follow

```
i = 0                                # loop counter
while <condition> :
    <statement(s)>
    i += 1
    if i > 1000 : break              # exit if 1000 loops
```

```
while <condition> :                  # user input
    <statement(s)>
    a = input('press any key to proceed')
    if a : break                    # user can interrupt at will
```

Functional programming

Functional programming idioms in Python

Functional programming is computing by **evaluating functions**

Includes processing items in an iterable without using a loop

The most common functional programming idioms in Python are:

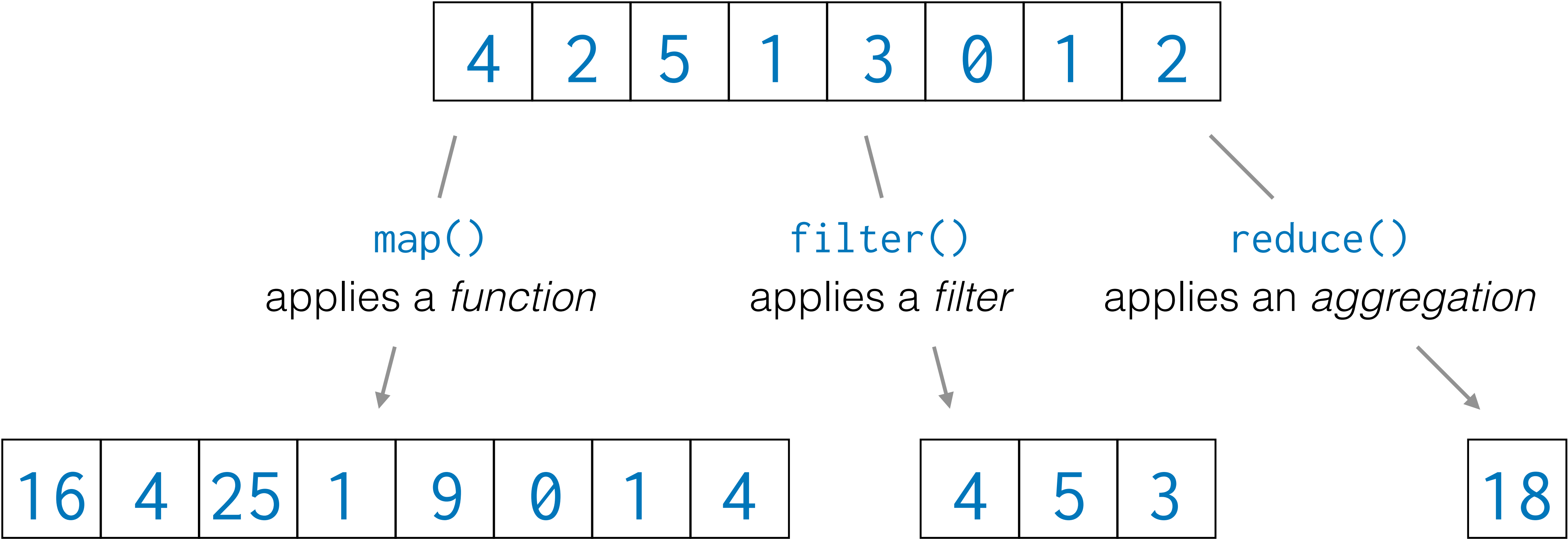
Implicit loops: apply an operation to each item

Lambda functions: single-use functions included within a statement

Comprehensions: compact and flexible syntax for implicit loops

Implicit loops with `map()`, `filter()`, and `reduce()`

These functions apply an operation to items in an ordered container (e.g., lists and tuples)



Implicit loops with `map()`

A standard for loop that applies a function to every item in a list looks like this:

```
verts = ['armadillo', 'bat', 'rhinoceros', 'cat', 'lemur' ]
processed = []
for i in verts :
    word_len = len(i)
    processed.append(word_len)
```

The functional programming equivalent using `map()` is much simpler:

```
verts = ['armadillo', 'bat', 'rhinoceros', 'cat', 'lemur' ]
processed = list(map(len, verts))    # omit '()' after the function name
```

Note: `map()` returns a map object, so wrap in `list()` to cast to list

Lambda functions

Lambda functions are single-use functions embedded within another expression

Also known as anonymous or temporary functions

They are most useful when a function is only needed once and is simple

Syntax

```
lambda <parameter list>: <expression>
```

Lambda functions are often used with `map()`, `filter()`, and `reduce()`

```
result = list(map(lambda x: x**2, input_list))      # square each number
result = list(filter(lambda x: x >= 3, input_list))  # filter numbers ≥ 3
result = list(reduce(lambda x, y: x+y, input_list)) # compute the overall sum
```

Comprehensions

Python provides a functional programming idiom to simplify **for** loops

Syntax is simpler and often easier to read, but reserve for simple conditions

Known as **comprehension**; can be applied to any iterable but most common with lists

A standard **for** loop that operates on a list looks like this:

```
fruits = ['banana', 'pineapple', 'mango', 'plum', 'avocado']
new_list = []
for i in fruits :
    if 'p' in i : new_list.append()
```

A list comprehension that does the same thing looks like this:

```
new_list = [i for i in fruits if 'p' in i]
```

Functions

Functions in Python



Functions can be treated as a “black box” from a logic perspective

All the programmer cares about is input and output

Specifics of how the transformation works is irrelevant to program logic

Contributes to abstraction and is a big help in designing and debugging a program

Input values

Technically

Parameters are stated in the function declaration

Arguments are passed to the parameters during run-time

Python (like R) is very flexible regarding parameters:

0, 1, or multiple

Required, optional, or both: use `=` to set default values for optional parameters

Fixed or variable number: use `*args` and `**kwargs` for variable number

And regarding arguments:

Any data object(s)

Any function(s)

Return values

By default, every function returns the value `None`

This behavior can be altered using a `return` statement

0, 1, or multiple objects can be returned; if more than one, packaged into a tuple

Best practice is to place the `return` statement at the end of the code block

Multiple `return` statements can be included when using if / else structures

Functions can return values, named data objects, or functions

When more than one item is returned, they are automatically placed into a tuple

Defining a function

Creating a function

Use the `def` keyword

Use colon and indenting rather than curly braces to define the code block

```
# create a function to test whether 7 is a factor
def is_div_by_7(input_value) :
    if input_value % 7 == 0 : return True
    else: return False
# call the function
is_div_by_7(4325427)      # returns False
is_div_by_7(7999243)      # returns True
```

Defining a function, continued

Functions must contain **at least one statement** in the code block

Use `pass` as a place-holder to fill the code block (does nothing during execution)

Use `return` statements to control what the function returns

If there is no `return` statement, the function returns the value `None`

Using `return` by itself also returns the value `None` (useful in if/else structures)

Using `return` followed by value(s) returns the value(s)

Use a **docstring** to provide a short description of the function and its use

Place in triple quotes on the line immediately following the function declaration

```
"""Takes a DNA string as input and returns codons as a list"""
```


Defining arguments for functions

Optional arguments are indicated with `=` followed by the default value

```
def function_name(arg1 = 0, arg2 = True) :  
    <statement(s)>
```

Optional arguments must follow all required arguments

The ability to pass an **arbitrary number of arguments** is indicated with `*args`

```
def function_name(*args) :  
    <statement(s)>
```

Represented internally as a tuple; access as `args` within the function

The ability to pass an **arbitrary number of keyword arguments** is indicated with `**kwargs`

```
def function_name(**kwargs) :  
    <statement(s)>
```

Represented internally as a dictionary; access as `kwargs` within the function

Passing arguments to functions

Separate multiple arguments with commas

First, pass any **required** arguments in the defined order

Anything else will throw an error

Next, pass any ***args** and ****kwargs** arguments

Last, pass any optional arguments

Order is not important

Omit unless changing the default value (including is innocuous)

Include the argument and value (e.g., `reverse = True`)

Scope

Global variables can be updated within a function

However, this will not alter the value of the global variable *outside* the function

Any variable defined within a function is assumed to be local to that function

To define a global variable within a function, use the `global` keyword (not recommended)

```
def my_func() :  
    global my_var      # first, declare the variable as global  
    my_var = 'spam'    # then, assign a value
```

To return all variables and values within a function, call `locals()` or `globals()`

```
def my_func() :  
    locals()           # returns a dictionary of local variables and values  
    globals()          # returns a dictionary of global variables and values
```


Evaluation

Functions are not evaluated when defined

Only evaluated when called by other code at runtime (known as **lazy evaluation**)

Default argument values are evaluated once, left to right, when the function is executed

Can cause unexpected behavior if a default value is altered within the function

Keep both points in mind when debugging code!

